

Laboratory 5:

LTI System Properties & Singularity Functions RC/RL Circuit Impulse & Step Responses

Measuring First-Order System Dynamics in Hardware

Lab Duration: 2 hours

Pre-Lab Time Expected: 30 minutes (theoretical derivations)

Key Concept: Understanding how real circuits respond to idealized inputs (step, impulse) and extracting time constants from experimental data.

Contents

1	Learning Objectives	6
2	Core Concepts: Singularity Functions and First-Order LTI Systems	7
2.1	What Are Singularity Functions?	7
2.2	First-Order LTI Systems	7
2.3	Step Response of First-Order Systems	7
2.4	Impulse Response of First-Order Systems	7
2.5	Measuring τ from Experimental Data	8
3	Required Equipment and Software	9
4	Safety and Critical Wiring Notes	9
5	Hardware Circuits: RC and RL Low-Pass Configurations	10
5.1	RC Low-Pass Circuit	10
5.2	RL Low-Pass Circuit	11
5.3	Impulse Approximation in Hardware	12
6	Pre-Lab Preparation (Complete Before Lab Session)	13
6.1	Component Measurement	13
6.2	Theoretical Derivations (Handwritten)	13
6.3	Analytical Step and Impulse Responses	14
6.4	Theoretical Time Constants	14
6.5	Pre-Lab Figure (MATLAB)	15
7	Lab Procedure (In-Lab Execution, 2 Hours)	16
7.1	Part 1: Build and Verify RC Low-Pass Circuit	16
7.2	Part 2: Build and Verify RL Low-Pass Circuit (with Flyback Diode)	17
7.3	Part 3: Arduino Software Configuration and Testing	18
7.4	Part 4: MATLAB Data Acquisition and Analysis Workflow	20
8	Post-Lab Analysis and Deliverables	24
8.1	Analysis Results Table	24

8.2	Required Discussion Questions	24
8.3	Final Submission Checklist	25
8.4	TA Checkpoint Sign-Off	26
Appendix A: Full Theory Derivations (Reference)		27
Appendix B: Arduino Configuration and Example Logic		29
Appendix C: MATLAB Analysis Templates and Code Snippets		30
Appendix D: Troubleshooting Guide		33
References and Inspiration		37

Grading and Authenticity Requirements

Deliverable (1/2): This is a graded laboratory assignment. Include your **name and student ID** in all required screenshots and photographs.

Deliverable (2/2): All curve fitting and MATLAB verification must use waveform data acquired from your own physical circuit. Pre-computed synthetic traces are not accepted.

Checkpoint (1/6): Before submission, verify that each MATLAB figure includes axis labels with units, a legend, and your name/student ID in the title (or `sgtitle`).

Title & Real-World Context

You are a Junior Biomedical Electronics Engineer at a Medical Device Design Firm

Your technical lead has tasked you with validating the analog signal conditioning pipeline for a **next-generation wearable patient monitoring system** that measures vital signs continuously in hospital and home-care settings. The system must reliably acquire three critical physiological measurements:

1. **Pulse Oximetry (Blood Oxygen Saturation).** Your device uses infrared LEDs to measure light absorption through tissue, generating weak analog signals (10–500 mV) that must be conditioned through a carefully tuned **RC filter** before ADC sampling. The time constant $\tau = RC$ determines the signal bandwidth: too fast (τ too small) and the filter misses low-frequency oxygen saturation trends; too slow (τ too large) and the system cannot track rapid changes during patient movement or alarm conditions. Clinical accuracy requires the step response to settle within 200 ms while suppressing 60 Hz AC mains noise. A miscalibrated time constant causes false hypoxia alarms that trigger unnecessary interventions or misses real desaturation episodes, jeopardizing patient safety.
2. **ECG (Electrocardiogram) Signal Acquisition.** Your wearable records heart electrical activity via chest electrodes, generating signals in the 0.5–100 Hz range with amplitudes of 1–5 mV. An **RL circuit** in the signal path models the electrode-skin interface impedance; its time constant $\tau = L/R$ must be optimized to avoid baseline drift (low-frequency wander from patient respiration) while preserving clinically important QRS complexes and T waves. Excessive RL time constant causes baseline oscillations that confuse rhythm detection algorithms; insufficient settling allows DC drift that saturates the ADC.
3. **Temperature Monitoring (Thermistor Conditioning).** A thermistor temperature sensor exhibits exponential charging behavior through an RC network when powered on; the system must accurately measure the transient response to infer the actual body temperature before the sensor reaches steady state. Time constant mismatch between the sensor and the calibration model causes systematic temperature errors that, when multiplied across thousands of patients, drives incorrect fever diagnosis and inappropriate antibiotic decisions in hospital networks.

Your Mission. In this lab, you will characterize and validate first-order RC and RL circuits by measuring their impulse and step responses:

- Derive the theoretical differential equations governing RC and RL transient behavior.
- Measure hardware impulse and step responses using Arduino and oscilloscope.
- Extract empirical time constants via log-linearization and curve fitting.
- Compare theory against measurement and quantify deviations due to real-world non-idealities.
- Validate that your circuits meet medical-device time-constant specifications.

The medical device industry certifies every component's time-domain behavior because physiological signals are fundamentally first-order phenomena: a patient's respiratory effort, cardiac rhythm, and temperature all evolve at timescales governed by RC time constants. Your rigorous characterization methodology—hand derivation, hardware measurement, curve fitting, error analysis—is the

engineering discipline that separates prototype breadboards from implantable and wearable devices that physicians trust with patient safety.

Constraints (Fixed). 2-hour lab execution window, existing hardware platform (Arduino Uno, oscilloscope, breadboard). All skeleton code provided. All checkpoints and deliverables remain exactly as specified herein.

1 Learning Objectives

Upon completion of this laboratory, you will demonstrate:

1. **Rigorous First-Order System Modeling from First Principles.** Derive the time-domain differential equations for RC and RL circuits using Kirchhoff's voltage law, identify the time constant τ as the fundamental parameter governing system speed, and predict analytical step and impulse responses in closed form.
2. **Hardware-Software Validation of Time-Domain Behavior.** Measure actual impulse and step responses on real circuits (accounting for Arduino output impedance, oscilloscope loading, parasitic resistance), acquire high-resolution ADC data, and compare empirical waveforms against theoretical predictions with quantified error metrics.
3. **Singularity Function Implementation and Approximation.** Understand the mathematical definitions of the Heaviside step $u(t)$ and Dirac delta $\delta(t)$; implement practical hardware approximations using Arduino PWM with configurable pulse width and timing; quantify the error introduced by finite rise/fall times and pulse duration.
4. **Time-Constant Extraction via Curve Fitting and Statistical Analysis.** Use log-linearization of exponential decay to extract empirical time constants $\hat{\tau}$ from noisy measurement data; apply linear regression with confidence intervals; compute percent error against theory; identify whether deviations are systematic (component tolerance, parasitic effects) or random (measurement noise).
5. **Production-Grade Non-Ideal Analysis.** Characterize and quantify real-world deviations from ideal first-order theory: inductor DC resistance, capacitor ESR (equivalent series resistance), ADC quantization, oscilloscope input loading, measurement delays, and component tolerances. Predict how these effects impact clinical medical device specifications (settling time, bandwidth, accuracy).
6. **Medical Device Time-Constant Specification and Validation.** Given clinical requirements (e.g., settle within 200 ms, suppress 60 Hz mains noise, preserve 0.5–100 Hz physiological signal), design an RC/RL filter with calculated component values, validate that measurements meet specifications, and propose component selection trade-offs suitable for production certification.

2 Core Concepts: Singularity Functions and First-Order LTI Systems

2.1 What Are Singularity Functions?

Singularity functions are idealized mathematical inputs used to characterize the behavior of systems. In this lab, we focus on two:

- **Heaviside Unit Step $u(t)$:** Represents an instantaneous switch from 0 to 1 at $t = 0$. Mathematically, $u(t) = 0$ for $t < 0$ and $u(t) = 1$ for $t \geq 0$. In hardware, we approximate this by setting a digital pin HIGH and keeping it there.
- **Dirac Delta Impulse $\delta(t)$:** A very narrow, very tall spike with unit area. Formally, $\delta(t) = 0$ everywhere except $t = 0$, and $\int_{-\infty}^{\infty} \delta(t) dt = 1$. In hardware, we approximate this with a short pulse of width $T_p \ll \tau$.

The response of a linear time-invariant (LTI) system to these inputs is fundamental:

- **Step response $s(t)$:** Output when $x(t) = V_s \cdot u(t)$ (a sudden voltage applied and held).
- **Impulse response $h(t)$:** Output when input is a narrow pulse that approximates $\delta(t)$.

2.2 First-Order LTI Systems

A first-order causal LTI system is fully described by the differential equation:

$$\tau \frac{dy(t)}{dt} + y(t) = Kx(t), \quad (1)$$

where:

- τ is the **time constant** (units: seconds), controlling response speed.
- K is the **DC gain**, the steady-state ratio of output to input.
- $x(t)$ is the input, $y(t)$ is the output.

Key insight: At time $t = \tau$, a rising exponential reaches 63.2% of steady state, and a decaying exponential falls to 36.8% of its initial value. The larger τ , the slower the system responds.

2.3 Step Response of First-Order Systems

For a unit step input $x(t) = V_s u(t)$, the analytical solution is:

$$y(t) = KV_s \left(1 - e^{-t/\tau}\right) u(t). \quad (2)$$

This equation tells us:

- At $t = 0$: $y(0) = 0$ (continuity for capacitive circuits).
- At $t = \tau$: $y(\tau) \approx 0.632 \cdot KV_s$ (63.2% of final value).
- As $t \rightarrow \infty$: $y(\infty) = KV_s$ (steady-state DC gain).

2.4 Impulse Response of First-Order Systems

If the input is an impulse $x(t) = K\delta(t)$, the impulse response is:

$$h(t) = \frac{K}{\tau} e^{-t/\tau} u(t). \quad (3)$$

Note that $h(t)$ starts at a peak value of K/τ and decays exponentially. The impulse response directly reveals the time constant through its decay rate.

2.5 Measuring τ from Experimental Data

When you measure noisy step-response data $y(t)$, you can extract τ by:

1. **Remove baseline:** Subtract the initial value to isolate the transient.
2. **For rising exponentials:** Transform the data as $\ln(V_\infty - y(t))$ and fit a line to the result. The slope m satisfies $\tau = -1/m$.
3. **For decaying exponentials:** Transform as $\ln(y(t))$ and fit a line. Again, $\tau = -1/m$.
4. **Quantify error:** Compare your measured $\hat{\tau}$ with the theoretical value: Percent Error = $\left| \frac{\hat{\tau} - \tau_{\text{theory}}}{\tau_{\text{theory}}} \right| \times 100\%$.

This log-linearization technique is powerful because it converts an exponential curve fit into a linear regression problem, which MATLAB handles robustly.

[INSERT DIAGRAM: Step Response vs Impulse Response of a First-Order System (showing 63.2% at $t = \tau$, decay curves for both, with annotations)]

3 Required Equipment and Software

Table 1. Required Hardware and Software

Category	Required Items
Controller	Arduino Uno (preferred) or Mega, USB cable
RC components	Bench-assigned resistor R , capacitor C
RL components	Bench-assigned resistor R_{ext} , inductor L with winding resistance R_L
Protection	Flyback diode (1N4148 or 1N4001), mandatory for RL circuit
Measurement	Bench multimeter (resistance and capacitance modes)
Prototyping	Breadboard and jumper wires
Software	Arduino IDE and MATLAB R2021a or later
Provided skeletons	<code>FastADC_Stream.ino</code> , <code>Lab05_Serial_Reader.m</code>

Arduino Pin Roles

Table 2. Pin Assignments

Signal	Pin	Direction	Purpose
Input source $x(t)$	D7	OUTPUT	Drives step/impulse-like input
Sense input	A0	INPUT	Measures circuit node voltage
Trigger indicator	D13	OUTPUT	LED toggles at acquisition start
Serial link	USB	TX/RX	115200 baud stream to MATLAB

4 Safety and Critical Wiring Notes

- Keep Arduino ADC input A0 within 0–5 V at all times.
- Never power the RL circuit before installing and verifying the flyback diode orientation.
- Disconnect USB power before rewiring between RC and RL circuits.
- Confirm a common ground between Arduino GND and breadboard ground rail.

Flyback diode orientation for RL setup:

- Diode anode $\rightarrow$ GND side of inductor.
- Diode cathode (striped end) $\rightarrow$ node between R_{ext} and L .

5 Hardware Circuits: RC and RL Low-Pass Configurations

5.1 RC Low-Pass Circuit

Physical Layout: The RC circuit is a simple first-order low-pass filter. The Arduino D7 pin (output) drives a resistor R . The voltage across the capacitor C (to ground) is the output measured by A0.

Circuit topology:

- D7 $\rightarrow$ Resistor $R \rightarrow$ Node (measurement point) $\rightarrow$ Capacitor $C \rightarrow$ GND.
- Arduino GND connected to the breadboard GND rail.
- A0 probe connected to the node between R and C .

Governing equation (derived from KVL):

$$RC \frac{dV_C}{dt} + V_C = x(t), \quad \text{where } \tau_{RC} = RC. \quad (4)$$

For a step input $x(t) = V_s u(t)$, the response is:

$$V_C(t) = V_s \left(1 - e^{-t/\tau_{RC}} \right) u(t). \quad (5)$$

For an impulse approximation (narrow pulse), the peak response is:

$$V_C^{\text{peak}} \approx \frac{V_s T_p}{RC}, \quad (6)$$

where V_s is the Arduino HIGH level (nominally 5 V) and T_p is the pulse width in seconds.

[INSERT RC CIRCUIT SCHEMATIC: Arduino D7 to R to Node (with A0 probe) to C to GND, with clear labels]

Student Notes: Measured Component Values (RC Circuit):

$R = \underline{\hspace{2cm}} \Omega$

$C = \underline{\hspace{2cm}} \mu\text{F}$

$\tau_{RC} = RC = \underline{\hspace{2cm}} \text{ ms}$

5.2 RL Low-Pass Circuit

Physical Layout: The RL circuit contains a series resistor R_{ext} and an inductor L with its own winding resistance R_L . A **flyback diode** is essential to prevent voltage spikes when the inductor current is interrupted.

Circuit topology:

- D7 $\rightarrow$ Resistor R_{ext} $\rightarrow$ Node (measurement point) $\rightarrow$ Inductor L (with winding resistance R_L) $\rightarrow$ GND.
- Flyback diode connected across the inductor: cathode (striped end) to the node between R_{ext} and L , anode to the inductor's GND side.
- A0 probe connected to the node between R_{ext} and L .

Governing equations:

The current $i(t)$ through the series R_{ext} and L satisfies:

$$L \frac{di}{dt} + (R_{\text{ext}} + R_L)i(t) = x(t), \quad (7)$$

where $R_{\text{eff}} = R_{\text{ext}} + R_L$ is the effective resistance.

The voltage across R_{ext} (the quantity of interest) is:

$$V_{R_{\text{ext}}}(t) = R_{\text{ext}} \cdot i(t). \quad (8)$$

However, **A0 measures the node voltage:**

$$V_{\text{node}}(t) = \text{voltage between } R_{\text{ext}} \text{ and } L. \quad (9)$$

Critical correction: To recover the voltage across the external resistor:

$$V_{R_{\text{ext}}}(t) = V_{\text{in}}(t) - V_{\text{node}}(t), \quad (10)$$

where $V_{\text{in}}(t)$ is the Arduino D7 voltage (approximately 5 V when HIGH).

The resulting first-order system has:

$$\tau_{RL} = \frac{L}{R_{\text{eff}}} = \frac{L}{R_{\text{ext}} + R_L}, \quad K = \frac{R_{\text{ext}}}{R_{\text{eff}}}. \quad (11)$$

For a step input $x(t) = V_s u(t)$:

$$V_{R_{\text{ext}}}(t) = K V_s \left(1 - e^{-t/\tau_{RL}}\right) u(t). \quad (12)$$

[INSERT RL CIRCUIT SCHEMATIC: Arduino D7 to R_{ext} to Node (with A0 probe and flyback diode across L) to L (with winding resistance noted) to GND, with clear labels]

Student Notes: Measured Component Values (RL Circuit):

$$R_{\text{ext}} = \text{_____} \Omega$$

$$L = \text{_____} \text{ mH}$$

$$R_L = \text{_____} \Omega \text{ (winding resistance)}$$

$$R_{\text{eff}} = R_{\text{ext}} + R_L = \text{_____} \Omega$$

$$\tau_{RL} = L/R_{\text{eff}} = \text{_____} \text{ ms}$$

$$K = R_{\text{ext}}/R_{\text{eff}} = \text{_____}$$

5.3 Impulse Approximation in Hardware

A finite pulse of width T_p (microseconds) can approximate the Dirac delta $\delta(t)$ if $T_p \ll \tau$. As a rule of thumb:

- If $T_p/\tau < 0.1$, the approximation is good.
- If $T_p/\tau \approx 0.5$, the pulse is too wide; the response will not look like a true impulse.

The TA will assign a specific pulse width T_p that is appropriate for your measured time constants.

6 Pre-Lab Preparation (Complete Before Lab Session)

6.1 Component Measurement

Pre-lab Deliverable (1/7): Every bench has uniquely labeled components. You and your partner must measure your specific resistor, capacitor, and inductor using the multimeter. Do **NOT** share digital values with another lab pair—the TA will verify your specific components. Your measurements must be recorded in writing before you enter the lab.

Table 3. Component Measurements (Record at Your Lab Bench)

Symbol	Description	Measured Value
R	RC resistor (measured with multimeter)	_____ Ω
C	RC capacitor (measured with multimeter)	_____ μF
R_{ext}	RL external resistor (measured)	_____ Ω
L	Inductor (measured with multimeter)	_____ mH
R_L	Inductor winding resistance (measured)	_____ Ω
R_{eff}	$R_{\text{ext}} + R_L$ (calculated)	_____ Ω
T_p	TA-assigned pulse width (given by TA)	_____ μs

6.2 Theoretical Derivations (Handwritten)

Pre-lab Deliverable (2/7): You must hand-derive the governing differential equations for both RC and RL circuits using Kirchhoff's voltage law (KVL). Show all steps.

RC Circuit Derivation:

Starting from KVL: $V_R(t) + V_C(t) = x(t)$, where $V_R = R \cdot i(t)$ and $i(t) = C \frac{dV_C}{dt}$.

Student Notes: Space for handwritten RC derivation:

Final RC Equation:

$$\tau_{RC} \frac{dV_C}{dt} + V_C = x(t), \quad \text{where} \quad \tau_{RC} = RC$$

RL Circuit Derivation:

Starting from KVL: $V_R(t) + V_L(t) = x(t)$, where $V_R = (R_{\text{ext}} + R_L) \cdot i(t)$ and $V_L = L \frac{di}{dt}$.

Student Notes: Space for handwritten RL derivation:

Final RL Equation (in terms of $V_{R_{\text{ext}}}$):

$$\tau_{RL} \frac{dV_{R_{\text{ext}}}}{dt} + V_{R_{\text{ext}}} = K \cdot x(t), \quad \text{where} \quad \tau_{RL} = \frac{L}{R_{\text{eff}}}, \quad K = \frac{R_{\text{ext}}}{R_{\text{eff}}}$$

6.3 Analytical Step and Impulse Responses

Pre-lab Deliverable (3/7): Write the analytical expressions for step response, impulse response, and expected impulse peak using your measured component values.

Student Notes: RC Step Response: $V_C(t) = \underline{\hspace{2cm}}$ ($t \geq 0$)

RC Impulse Response: $h(t) = \underline{\hspace{2cm}}$ ($t \geq 0$)

RL Step Response: $V_{R_{\text{ext}}}(t) = \underline{\hspace{2cm}}$ ($t \geq 0$)

RL Impulse Response: $h(t) = \underline{\hspace{2cm}}$ ($t \geq 0$)

Expected impulse peak (RC): $\approx \underline{\hspace{2cm}}$ V

Expected impulse peak (RL): $\approx \underline{\hspace{2cm}}$ V

6.4 Theoretical Time Constants

Pre-lab Deliverable (4/7): Calculate τ_{RC} and τ_{RL} numerically using your measured component values.

Student Notes: Calculated Time Constants:

$$\tau_{RC} = R \times C = \underline{\hspace{2cm}} \times \underline{\hspace{2cm}} = \underline{\hspace{2cm}} \text{ ms}$$

$$\tau_{RL} = L/R_{\text{eff}} = \underline{\hspace{2cm}} / \underline{\hspace{2cm}} = \underline{\hspace{2cm}} \text{ ms}$$

6.5 Pre-Lab Figure (MATLAB)

Pre-lab Deliverable (5/7): Generate a MATLAB figure with 4 subplots showing theoretical step and impulse responses for both RC and RL circuits. Use your calculated τ values and assume a 5 V input.

Student Notes: Student notes on pre-lab theory figure:

Checkpoint (2/6): Before entering the lab, show the TA your **handwritten derivations** (with name/student ID), your **calculated time constants**, and your **pre-lab MATLAB figure** for Checkpoint 1 sign-off.

7 Lab Procedure (In-Lab Execution, 2 Hours)

Target Timeline: CP-1 (15 min) → RC wiring (20 min) → RL wiring (20 min) → Arduino testing (15 min) → MATLAB analysis (45 min) → wrap-up (5 min).

7.1 Part 1: Build and Verify RC Low-Pass Circuit

Objective: Construct the RC circuit, verify connectivity, and confirm that the Arduino can acquire voltage data as the capacitor charges and discharges.

1. **Disconnect USB.** Ensure the Arduino is not powered.
2. **Breadboard wiring:** On your breadboard, create the signal path:
 - Connect Arduino D7 (OUTPUT pin) to one end of resistor R .
 - Connect the other end of resistor R to a node on the breadboard. This is your **measurement node**.
 - Connect one leg of capacitor C to this measurement node.
 - Connect the other leg of capacitor C to the breadboard GND rail.
 - Connect the breadboard GND rail to Arduino GND.
3. **Connect A0 probe:** Attach the analog input pin A0 to the measurement node (between R and C).
4. **Verify connectivity:** Before powering, trace the current path with your finger:
 - $D7 \rightarrow R \rightarrow \text{measurement node} \rightarrow C \rightarrow \text{GND} \rightarrow \text{Arduino GND} \rightarrow \text{back to } D7$.
 - Confirm there is one complete loop with no breaks.
5. **No components touching GND or power rails directly.** Each component should be inserted into separate breadboard holes.

**[INSERT RC CIRCUIT WIRING PHOTO or SCHEMATIC
for breadboard implementation with labels]**

Student Notes: Student checklist for RC wiring:

- ☐ Current path verified: $D7 \rightarrow R \rightarrow C \rightarrow GND$
- ☐ A0 connected to node between R and C
- ☐ Arduino GND connected to breadboard GND
- ☐ No floating or unconnected wires

Checkpoint (3/6): After wiring, do NOT plug in USB yet. Wait for TA visual inspection before proceeding.

7.2 Part 2: Build and Verify RL Low-Pass Circuit (with Flyback Diode)

Objective: Reconfigure the breadboard for the RL circuit, paying special attention to the flyback diode placement and orientation.

WARNING: Inductors can generate large voltage spikes when current is interrupted. The flyback diode is critical to prevent Arduino damage.

1. **Disconnect USB and power off the Arduino.**
2. **Remove the RC components:** Carefully remove resistor R and capacitor C from the breadboard, leaving the breadboard wiring intact for the new circuit.
3. **Install resistor R_{ext} :** Wire in the same position as R was in the RC circuit:
 - $D7 \rightarrow R_{\text{ext}} \rightarrow$ measurement node.
4. **Install inductor L :** Connect L from the measurement node to GND (in the same position as C was).
5. **Install flyback diode (CRITICAL):**
 - Diode cathode (striped end) $\rightarrow$ measurement node (between R_{ext} and L).
 - Diode anode (unmarked end) $\rightarrow$ GND side of L .
 - **Polarity is critical:** An incorrectly oriented diode will not protect the circuit and may cause Arduino damage.
6. **A0 remains connected** to the measurement node (between R_{ext} and L).
7. **Verify the complete path:**
 - $D7 \rightarrow R_{\text{ext}} \rightarrow$ measurement node $\rightarrow L \rightarrow GND \rightarrow$ Arduino GND.
8. **Diode orientation check:** Using a multimeter in diode mode, confirm the diode conducts only in the reverse direction (from GND to node). If it conducts in the forward direction too, it is installed backwards.

[INSERT RL CIRCUIT WIRING PHOTO or SCHEMATIC showing flyback diode placement with clear polarization labels]

Student Notes: Student checklist for RL wiring:

- ☐ Current path verified: D7 $\rightarrow$ Rext $\rightarrow$ L $\rightarrow$ GND
- ☐ A0 connected to node between Rext and L
- ☐ Flyback diode installed: cathode (stripe) to node, anode to GND
- ☐ Diode orientation verified with multimeter
- ☐ Arduino GND connected to breadboard GND

Checkpoint (4/6): TA must visually inspect the flyback diode orientation and node wiring before USB is connected. Do not power up without TA approval.

7.3 Part 3: Arduino Software Configuration and Testing

Objective: Upload the provided `FastADC_Stream.ino` skeleton code, configure timing parameters, and verify that data is being acquired and transmitted to MATLAB.

Software Overview

The `FastADC_Stream.ino` skeleton handles:

- High-speed ADC sampling at rates up to 5–10 kHz (depending on your selected sample rate).
- Time-stamped acquisition: each ADC reading is paired with a microsecond timestamp.
- Serial transmission of data at 115200 baud to MATLAB.
- Simple input/output control: set D7 HIGH for a step, or pulse it for an impulse.

You do not need to modify the core ADC acquisition logic. Only edit the configuration section.

Configuration Parameters

Open `FastADC_Stream.ino` and locate the `// USER CONFIGURATION` section. Edit these constants:

- `SAMPLE_RATE_HZ`: Set to at least $10 \times 1/\tau_{\min}$ (where $\tau_{\min}$ is your smallest time constant). For example, if $\tau = 1$ ms, use $f_s \geq 10$ kHz. A higher sample rate is better (less aliasing risk) but

requires more MATLAB memory. Suggested: 5000–10000 Hz.

- **N_SAMPLES:** Number of samples to acquire per burst. To capture at least 5 time constants, set $N_{\text{samples}} \geq 5 \times \tau / \Delta t$, where $\Delta t = 1/f_s$. For example, if $\tau = 1$ ms and $f_s = 5$ kHz, then $N_{\text{samples}} \geq 5 \times 0.001 / 0.0002 = 25000$ samples.
- **PULSE_WIDTH_US:** TA-assigned impulse pulse width (microseconds). Typical values: 50–500 μs .
- **INPUT_TYPE:** Select **STEP** for a sustained HIGH signal, or **IMPULSE** for a narrow pulse.

Student Notes: Arduino configuration parameters (from TA or calculated):

SAMPLE_RATE_HZ = _____ Hz

N_SAMPLES = _____

PULSE_WIDTH_US = _____ μs

Acquisition length = _____ ms (= N_SAMPLES / SAMPLE_RATE_HZ)

Uploading and Testing

1. **Compile and upload** `FastADC_Stream.ino` to the Arduino using the Arduino IDE.
2. **Open Serial Monitor** (Ctrl+Shift+M) and set the baud rate to 115200.
3. **Trigger an acquisition:** In the Serial Monitor, type `s` and press Enter. The Arduino will:
 - Flash the LED on D13 to indicate acquisition start.
 - Apply the configured input (step or impulse) on D7.
 - Acquire N_SAMPLES readings from A0.
 - Stream data to the Serial Monitor in the format: `timestamp_us,adc_count`.
4. **Verify data is streaming:** Look for lines like:

```
0,342
200,356
400,371
600,387
...
```

The timestamp increases, and the ADC count should **change over time** (not stay constant). If the values are constant, check A0 wiring or component values.

5. **Collect RC step and impulse data:** Acquire one burst with `INPUT_TYPE = STEP` and one with `INPUT_TYPE = IMPULSE`. Copy the serial output for use in MATLAB.

6. **Reconfigure for RL circuit:** Switch the wiring to the RL circuit on the breadboard, then re-acquire two more bursts (step and impulse).

Checkpoint (5/6): Show TA a Serial Monitor screenshot with at least 10 lines of changing ADC data for both RC and RL circuits.

Student Notes: Student notes from Serial Monitor testing:

7.4 Part 4: MATLAB Data Acquisition and Analysis Workflow

Objective: Import Arduino serial data into MATLAB, convert ADC counts to voltages, overlay measured data against theoretical predictions, extract empirical time constants, and quantify error.

Template: Use the provided `Lab05_Serial_Reader.m` skeleton. The skeleton handles serial communication and raw data import. You fill in the **analysis blocks**.

Workflow Overview

1. **Data Import:** The skeleton script opens the serial port, triggers an Arduino acquisition (sends 's'), and reads the incoming data stream.
2. **Conversion:** Convert ADC counts (0–1023) to voltages (0–5 V): $V = (\text{ADC count})/1023 \times 5.0 \text{ V}$.
3. **Task 1 – Overlay:** Plot measured voltage and overlay your theoretical prediction.
4. **Task 2 – Curve Fit:** Extract the empirical time constant $\hat{\tau}$ using log-linearization.
5. **Task 3 – Error:** Calculate percent error and summarize results in a table.

Task 1: Overlay Measured Data vs Theoretical Prediction

For each of the four waveforms (RC step, RC impulse, RL step, RL impulse), create a figure with two subplots:

Top subplot (time domain):

- Plot `time_s` (x-axis, in seconds) versus `voltage_V` (y-axis, measured data) as a blue trace with no line (use `'-'` or `'.'`).
- Overlay your theoretical prediction (solid red line) using the equations from Section 2 or Appendix A.
- Add axis labels: "Time (s)" and "Voltage (V)".
- Add a legend: "Measured" and "Theory".

- Set the title to include your **name, student ID, circuit type, and input type**, e.g., "Alice Smith (A123) - RC Circuit Step Response".

Important correction for RL circuits:

- A0 measures the voltage at the node between R_{ext} and L (the “left side” of the inductor). This is not the voltage across R_{ext} .
- To compute the voltage **across** R_{ext} , use Kirchhoff’s voltage law:

$$V_{R_{\text{ext}}}(t) = V_{\text{in}}(t) - V_{\text{node}}(t) = 5.0 - V_{\text{node}}(t). \quad (13)$$

- Use this **corrected** $V_{R_{\text{ext}}}(t)$ trace for plotting and curve fitting in the RL analysis.

Student Notes: Student notes on overlay creation:

[INSERT EXAMPLE MATLAB FIGURE: Measured vs Theoretical RC Step Response overlay]

Task 2: Extract Empirical Time Constant via Log-Linearization

Goal: Fit an exponential to the measured data and extract $\hat{\tau}$.

For step responses (rising curves):

1. **Find steady-state value:** Compute the average of the last 10% of your data. Call this V_{∞} .
2. **Select a fitting window:** Choose a time range where transient dynamics dominate (e.g., 0 to 5τ). Avoid the very beginning (where dV/dt is largest and noise dominates) and the very end (where noise matters more).
3. **Transform the data:**

$$y_{\text{transform}} = V_{\infty} - V_{\text{measured}}. \quad (14)$$

This converts a rising curve into a decaying curve.

4. **Apply natural logarithm:**

$$y_{\log} = \ln(y_{\text{transform}}). \quad (15)$$

5. **Linear regression:** Use MATLAB's `polyfit(time, y_log, 1)` to fit a line $y_{\log} = m \cdot t + b$. The slope m satisfies:

$$m = -\frac{1}{\tau}, \quad \text{so} \quad \hat{\tau} = -\frac{1}{m}. \quad (16)$$

6. **Plot diagnostic figure:**

- **Top subplot:** Original measured signal + overlaid fitted exponential $Ae^{-t/\hat{\tau}}$.
- **Bottom subplot:** Log-domain data ($\ln(V_{\infty} - V)$ vs t) with the fitted line overlaid.

For impulse responses (decaying curves):

1. **Remove baseline:** If the signal doesn't decay to zero (e.g., noise offset), subtract the final value.
2. **Select fitting window:** Typically from the peak to $\approx 5\tau$ after the peak.
3. **Apply log directly:**

$$y_{\log} = \ln(y_{\text{measured}} - y_{\text{baseline}}). \quad (17)$$

4. **Linear regression:** Fit a line and extract $\hat{\tau} = -1/m$ as before.

Student Notes: Student log-linearization notes:

[INSERT EXAMPLE MATLAB FIGURE: Diagnostic plot showing original curve and log-domain fit]

Task 3: Compute and Report Percent Error

Pre-lab Deliverable (6/7): For all four waveforms, calculate:

$$\text{Percent Error} = \left| \frac{\hat{\tau}_{\text{measured}} - \tau_{\text{theory}}}{\tau_{\text{theory}}} \right| \times 100\%. \quad (18)$$

Organize results in a MATLAB table or printed summary (see Section 8 below). Include:

- Waveform type (e.g., “RC Step”, “RL Impulse”).
- Theoretical τ (from pre-lab calculation).
- Measured $\hat{\tau}$ (from log-linearization).
- Percent error.
- Brief notes on whether the error is acceptable (typically $< 20\%$ for well-designed hardware).

Checkpoint (6/6): Show the TA all four overlay figures, all four diagnostic fit figures, and your percent-error summary table for Checkpoint 3 sign-off.

8 Post-Lab Analysis and Deliverables

8.1 Analysis Results Table

Complete this table with your measured time constants and error calculations. Print this page and bring it to the submission checkpoint.

Table 4. Time-Constant Measurements and Error Analysis

Waveform	τ_{theory} (ms)	$\hat{\tau}_{\text{measured}}$ (ms)	Error (ms)	Error (%)
RC step response	_____	_____	_____	_____
RC impulse response	_____	_____	_____	_____
RL step response	_____	_____	_____	_____
RL impulse response	_____	_____	_____	_____

Acceptable error range: Typically $< 20\%$ for well-designed circuits. Errors $> 30\%$ should prompt investigation (see Troubleshooting in Appendix D).

8.2 Required Discussion Questions

Pre-lab Deliverable (7/7): Answer each question in 3–6 complete sentences, supported by numerical evidence from your data. Attach this to your submission.

- Energy Storage Comparison (RC vs RL):** Explain the fundamental difference between how RC and RL circuits store energy. Why does the RL circuit's DC gain ($K = R_{\text{ext}}/R_{\text{eff}}$) depend on inductor winding resistance R_L , while the RC circuit's DC gain is 1? How does this relate to the steady-state behavior you measured?
- Impulse Approximation Quality:** Calculate the ratio T_p/τ for both your RC and RL circuits using your TA-assigned pulse width T_p . Based on the rule of thumb ($T_p/\tau < 0.1$ for good approximation), evaluate whether your measured impulse responses match the theoretical impulse response $h(t) = (K/\tau)e^{-t/\tau}$.
- Log-Domain Nonlinearity:** In the log-linearization plots (Task 2), did your log-domain data form a perfectly straight line, or did it curve? Identify at least two physical sources of curvature (e.g., noise, finite transient response, baseline drift) and propose a mitigation strategy for each.
- Component Tolerance Analysis:** Using typical resistor tolerances (e.g., $\pm 5\%$) and capacitor tolerances (e.g., $\pm 10\%$), compute the worst-case bounds for τ_{RC} . Does your measured $\hat{\tau}_{RC}$ fall within these bounds? If not, what additional error sources might explain the difference?
- Impulse vs Step Consistency:** For each circuit (RC and RL), you measured both step and impulse responses. Theoretically, the impulse response should be the time derivative of the step response. Examine your measured data: do the impulse-response peak value and decay time roughly match those predicted by $h(t) = (K/\tau)e^{-t/\tau}$, or do they deviate? Explain discrepancies.

Student Notes: Space for discussion answers (or attach separate sheet with name/ID):

8.3 Final Submission Checklist

Before submitting, verify that your submission includes **all of the following**:

- ☐ **Pre-lab materials:** Handwritten derivations (RC and RL) with name/ID, measured component values, and calculated theoretical τ values.
- ☐ **Pre-lab MATLAB figure:** One 2×2 subplot figure showing theoretical step and impulse predictions for RC and RL circuits (from pre-lab).
- ☐ **Circuit photographs:** Clear photos of RC and RL breadboard setups with your name/ID visible and labeled measurement nodes. RL photo must show flyback diode orientation clearly.
- ☐ **Serial Monitor evidence:** Screenshot showing at least 10 lines of streamed timestamp and ADC data from the Arduino (demonstrating successful high-speed acquisition).
- ☐ **Four overlay figures:** MATLAB plots showing measured voltage and theoretical curves for RC step, RC impulse, RL step, and RL impulse. Each figure must have axis labels (with units), legend, and your name/ID in the title.
- ☐ **Four diagnostic fit figures:** For each of the four waveforms, a 2-subplot figure showing (top) original measured signal with fitted exponential, and (bottom) log-domain data with linear regression line.
- ☐ **Results table:** Printed/typed table (Table 4) with τ_{theory} , $\hat{\tau}_{\text{measured}}$, error, and percent error for all four waveforms.
- ☐ **Discussion answers:** Written responses to the five discussion questions with numerical support.

- **MATLAB code appendix:** Print or screenshot the key MATLAB code used for data import, overlay creation, and curve fitting. Include comments explaining each major block.
- **TA sign-off page:** See Table 5 below, completed with TA initials at each checkpoint.

8.4 TA Checkpoint Sign-Off

Table 5. TA Checkpoint Sign-Off (Complete as Lab Progresses)

CP	Requirement	Target Time	TA Initials & Date
CP-1	Pre-lab derivations, measured component values, calculated τ values, and pre-lab theory figure verified.	0:00–0:15	_____
CP-2	RC breadboard wiring verified. RL breadboard wiring verified. Flyback diode orientation confirmed and correct. Arduino high-speed data streaming verified in Serial Monitor.	0:15–1:05	_____
CP-3	All four overlay figures complete and accurate (measured vs theory). All four diagnostic fit figures complete with log-domain regression. Results table filled with percent errors.	1:05–1:50	_____

Lab Complete: Once all three checkpoints are signed off and the final submission checklist is complete, your lab is done. Congratulations!

Appendix A: Full Theory Derivations (Reference)

A.1 RC Low-Pass Circuit Derivation

Setup: A resistor R in series with a capacitor C to ground, with input $x(t)$ applied across the series combination. Output is the voltage $V_C(t)$ across the capacitor.

Kirchhoff's Voltage Law (KVL):

$$x(t) = V_R(t) + V_C(t), \quad (19)$$

where $V_R(t) = R \cdot i(t)$ is the resistor voltage and $i(t) = C \frac{dV_C}{dt}$ is the capacitor current.

Substitution:

$$x(t) = R \cdot C \frac{dV_C}{dt} + V_C(t). \quad (20)$$

Standard form:

$$RC \frac{dV_C}{dt} + V_C(t) = x(t), \quad \text{where } \tau_{RC} = RC. \quad (21)$$

Step Response ($x(t) = V_s u(t)$, a constant voltage V_s applied at $t = 0$):

Solving the first-order ODE with initial condition $V_C(0) = 0$:

$$V_C(t) = V_s \left(1 - e^{-t/\tau_{RC}}\right) u(t). \quad (22)$$

At $t = \tau_{RC}$, the capacitor has charged to $V_s(1 - e^{-1}) \approx 0.632V_s$ (63.2% of the final value).

Impulse Response (input is $\delta(t)$, the Dirac impulse):

For an idealized impulse, the impulse response is:

$$h_{RC}(t) = \frac{1}{\tau_{RC}} e^{-t/\tau_{RC}} u(t) = \frac{1}{RC} e^{-t/(RC)} u(t). \quad (23)$$

Note that $h(0) = 1/\tau_{RC}$ is the initial peak, and the signal decays with time constant τ_{RC} .

A.2 RL Low-Pass Circuit Derivation

Setup: A resistor R_{ext} in series with an inductor L (which has winding resistance R_L). Input $x(t)$ is applied across the series combination. Output is the voltage $V_{R_{\text{ext}}}(t)$ across the external resistor.

Kirchhoff's Voltage Law (KVL):

$$x(t) = V_{R_{\text{ext}}}(t) + V_L(t), \quad (24)$$

where:

- $V_{R_{\text{ext}}}(t) = R_{\text{ext}} \cdot i(t)$ (voltage across the external resistor).
- $V_L(t) = L \frac{di}{dt} + R_L \cdot i(t)$ (voltage across the inductor, including its winding resistance).

Combined:

$$x(t) = R_{\text{ext}} \cdot i(t) + L \frac{di}{dt} + R_L \cdot i(t) = (R_{\text{ext}} + R_L) i(t) + L \frac{di}{dt}. \quad (25)$$

Define effective resistance:

$$R_{\text{eff}} = R_{\text{ext}} + R_L. \quad (26)$$

Rearrange:

$$L \frac{di}{dt} + R_{\text{eff}} i(t) = x(t). \quad (27)$$

Express in terms of output $V_{R_{\text{ext}}} = R_{\text{ext}} \cdot i$:

Taking the derivative: $i = V_{R_{\text{ext}}}/R_{\text{ext}}$ and $\frac{di}{dt} = \frac{1}{R_{\text{ext}}} \frac{dV_{R_{\text{ext}}}}{dt}$.

Substituting:

$$L \frac{1}{R_{\text{ext}}} \frac{dV_{R_{\text{ext}}}}{dt} + R_{\text{eff}} \frac{V_{R_{\text{ext}}}}{R_{\text{ext}}} = x(t). \quad (28)$$

Multiply through by R_{ext} :

$$L \frac{dV_{R_{\text{ext}}}}{dt} + R_{\text{eff}} V_{R_{\text{ext}}} = R_{\text{ext}} x(t). \quad (29)$$

Divide by R_{eff} to get standard form:

$$\frac{L}{R_{\text{eff}}} \frac{dV_{R_{\text{ext}}}}{dt} + V_{R_{\text{ext}}} = \frac{R_{\text{ext}}}{R_{\text{eff}}} x(t). \quad (30)$$

Standard form:

$$\tau_{RL} \frac{dV_{R_{\text{ext}}}}{dt} + V_{R_{\text{ext}}} = K \cdot x(t), \quad \text{where} \quad \tau_{RL} = \frac{L}{R_{\text{eff}}}, \quad K = \frac{R_{\text{ext}}}{R_{\text{eff}}}. \quad (31)$$

Note: The DC gain $K < 1$ because some of the input voltage is dropped across the inductor's winding resistance R_L .

Step Response ($x(t) = V_s u(t)$):

$$V_{R_{\text{ext}}}(t) = K V_s \left(1 - e^{-t/\tau_{RL}}\right) u(t) = \frac{R_{\text{ext}}}{R_{\text{ext}} + R_L} V_s \left(1 - e^{-t/\tau_{RL}}\right) u(t). \quad (32)$$

Impulse Response:

$$h_{RL}(t) = \frac{K}{\tau_{RL}} e^{-t/\tau_{RL}} u(t) = \frac{R_{\text{ext}}}{L} e^{-t/\tau_{RL}} u(t). \quad (33)$$

A.3 Log-Linearization for Time-Constant Extraction

Goal: Extract the time constant τ from experimental step or impulse response data.

For step/rising response $y(t) = Y_{\infty}(1 - e^{-t/\tau})$:

Transform: Let $z(t) = Y_{\infty} - y(t) = Y_{\infty} e^{-t/\tau}$.

Take natural log:

$$\ln(z) = \ln(Y_{\infty}) - \frac{t}{\tau}. \quad (34)$$

This is a line with slope $m = -1/\tau$. Fit with `polyfit(time, log(z), 1)` and extract $\hat{\tau} = -1/m$.

For impulse/decaying response $y(t) = Ae^{-t/\tau}$:

Take natural log directly:

$$\ln(y) = \ln(A) - \frac{t}{\tau}. \quad (35)$$

Again, fit a line and extract $\hat{\tau} = -1/m$.

Appendix B: Arduino Configuration and Example Logic

B.1 FastADC_Stream.ino Configuration Section

The provided skeleton code has a clear configuration section at the top. **Only edit the values in this section; do not modify the core ADC or serial routines.**

```
//===== USER CONFIGURATION SECTION =====
#define OUT_PIN      7          // D7: output pin (step/impulse driver)
#define ADC_PIN      A0         // A0: input pin (voltage sensor)
#define LED_PIN      13         // D13: status LED
#define SAMPLE_RATE_HZ 5000     // Samples per second (Hz)
#define N_SAMPLES     5000      // Total samples to acquire per burst
#define PULSE_WIDTH_US 500      // Pulse width for IMPULSE mode (microseconds)
#define INPUT_TYPE     STEP      // STEP or IMPULSE
//=====
```

How to set these parameters:

- **SAMPLE_RATE_HZ:** Choose a rate such that $f_s \geq 10/\tau_{\min}$. This ensures you capture at least 10 samples per time constant (Nyquist-like criterion). For $\tau_{\min} = 1$ ms, use $f_s \geq 10$ kHz. For $\tau_{\min} = 10$ ms, use $f_s \geq 1$ kHz.
- **N_SAMPLES:** Set so that the total acquisition time is at least $5\tau_{\max}$ (where $\tau_{\max}$ is your larger time constant). Acquisition time = N_{samples}/f_s . For example: $N_{\text{samples}} = 5000$ samples at $f_s = 5000$ Hz gives an acquisition of 1 second, which captures ≈ 100 time constants if $\tau = 10$ ms.
- **PULSE_WIDTH_US:** TA will specify. Typical range: 50–500 μs .
- **INPUT_TYPE:** Change to **IMPULSE** to test impulse response. Leave as **STEP** for step response.

B.2 Serial Protocol

The Arduino operates as follows:

1. **Waits for trigger:** After upload, the Arduino waits for a character on the serial port.
2. **User sends 's':** In the Arduino Serial Monitor, type **s** and press Enter. The Arduino will:
 - Flash the LED on D13.
 - Apply the configured signal (step or narrow pulse) on D7.
 - Acquire **N_SAMPLES** readings from A0 at the specified sample rate.
 - Stream data to the serial port.
3. **Data format:** Each line contains `timestamp_us,adc_count`, e.g.:

```
0,512
200,524
400,538
600,556
...
```

where `timestamp_us` is in microseconds (elapsed time since start of acquisition) and `adc_count` is the raw 10-bit ADC reading (0–1023).

4. **Conversion to voltage:** In MATLAB, convert ADC counts to volts using the Arduino reference voltage (nominally 5.0 V):

$$V = (\text{ADC count})/1023 \times 5.0 \text{ V}. \quad (36)$$

B.3 Example Configuration for This Lab

For typical student-measured values ($\tau_{RC} \approx 1 \text{ ms}$, $\tau_{RL} \approx 10 \text{ ms}$):

```
#define SAMPLE_RATE_HZ 10000      // 10 kHz for good time resolution
#define N_SAMPLES      50000      // 5 seconds at 10 kHz (covers 500 time constants)
#define PULSE_WIDTH_US 100        // 100 microseconds (< 0.1 * tau_min)
#define INPUT_TYPE      STEP      // Start with step
```

With these settings, you capture from $t = 0$ to $t = 5$ seconds, which gives:

- For $\tau_{RC} = 1 \text{ ms}$: 5000 time constants (plenty of data).
- For $\tau_{RL} = 10 \text{ ms}$: 500 time constants (still excellent coverage).

Appendix C: MATLAB Analysis Templates and Code Snippets

C.1 Data Import and Conversion

The provided `Lab05_Serial_Reader.m` skeleton handles serial communication. Once you have raw data (`timestamp_us` and `adc_count`), convert to engineering units:

```
% Raw data from Arduino
timestamp_us = data(:, 1);           % in microseconds
adc_count = data(:, 2);              % 10-bit ADC (0-1023)

% Convert to seconds and volts
time_s = timestamp_us / 1e6;         % convert to seconds
V_in_ref = 5.0;                     % Arduino 5V reference
voltage_V = (adc_count / 1023) * V_in_ref; % ADC to volts (0-5V)

% Remove initial offset (optional, improves log-fit)
V_offset = mean(voltage_V(1:10));    % average of first 10 samples
voltage_V = voltage_V - V_offset;
```

C.2 Theoretical Curve Generation

For step response:

```
% RC step response:  $V_C(t) = V_s * (1 - \exp(-t/\tau))$ 
tau_theory = tau_RC;           % or tau_RL
V_s = 5.0;                     % step amplitude
V_theory = V_s * (1 - exp(-time_s / tau_theory));
```

```
% RL step response includes DC gain K
K_RL = R_ext / (R_ext + R_L);
V_theory = K_RL * V_s * (1 - exp(-time_s / tau_theory));
```

For impulse response:

```
% RC impulse:  $h(t) = (1/\tau) * \exp(-t/\tau)$ 
h_theory = (1 / tau_theory) * exp(-time_s / tau_theory);
```

```
% RL impulse includes DC gain
h_theory = (K_RL / tau_theory) * exp(-time_s / tau_theory);
```

C.3 RL Node-to-Resistor Voltage Conversion

Critical step for RL analysis:

The Arduino A0 measures the voltage at the node between R_{ext} and L . To recover the voltage across R_{ext} , use KVL:

```
% A0 measures node voltage V_node
V_in = 5.0;                               % Arduino HIGH level during step
V_Rext = V_in - V_node;                   % voltage across R_ext
```

```
% For RL step response, use V_Rext for curve fitting, not V_node
%  $V_{\text{Rext}}(t) = K * V_s * (1 - \exp(-t/\tau_{\text{RL}}))$ 
```

```
% For RL impulse response:
%  $h(t) = K/\tau * \exp(-t/\tau)$ 
```

C.4 Log-Linearization for Time-Constant Extraction

For a rising step response:

```
% Step response:  $y(t) = V_s(1 - \exp(-t/\tau))$ 
% Transform:  $z(t) = V_s - y(t) = V_s * \exp(-t/\tau)$ 
% Log:  $\ln(z) = \ln(V_s) - t/\tau$ 
% Fit:  $\ln(z) = m*t + b$ , where  $m = -1/\tau$ 
```

```
% Compute steady state
V_ss = mean(voltage_V(end-100:end));      % average of last 100 samples
```

```
% Transform (rising to decay)
z = V_ss - voltage_V;
z = z(z > 0.01);                          % keep only positive values (noise mitigation)
time_fit = time_s(1:length(z));           % corresponding times
```

```
% Log transform
```

```
z_log = log(z);

% Linear regression
coeffs = polyfit(time_fit, z_log, 1);    % fit line: z_log = m*t + b
m = coeffs(1);                          % slope
tau_measured = -1 / m;                  % extract tau

% For validation, compute fitted curve
z_fitted = exp(coeffs(1)*time_fit + coeffs(2));
V_fitted = V_ss - z_fitted;

For a decaying impulse response:

% Impulse response:  $y(t) = A * \exp(-t/\tau)$ 
% Log:  $\ln(y) = \ln(A) - t/\tau$ 
% Fit:  $\ln(y) = m*t + b$ , where  $m = -1/\tau$ 

% Remove baseline offset
y_offset = mean(voltage_V(end-100:end));
y_centered = voltage_V - y_offset;
y_centered = y_centered(y_centered > 0.01); % keep positive values
time_fit = time_s(1:length(y_centered));

% Log transform
y_log = log(y_centered);

% Linear regression
coeffs = polyfit(time_fit, y_log, 1);
m = coeffs(1);
tau_measured = -1 / m;

% Fitted curve
y_fitted = exp(coeffs(1)*time_fit + coeffs(2));
```

C.5 Percent Error Calculation

```
% Given theoretical tau and measured tau
tau_theory = 1e-3;          % example: 1 ms
tau_measured = 1.05e-3;    % example: 1.05 ms

% Compute error
error_absolute = abs(tau_measured - tau_theory);
error_percent = (error_absolute / tau_theory) * 100;

fprintf('Theoretical tau: %.3f ms\n', tau_theory*1e3);
fprintf('Measured tau:    %.3f ms\n', tau_measured*1e3);
fprintf('Absolute error:  %.3f ms\n', error_absolute*1e3);
fprintf('Percent error:   %.1f %%\n', error_percent);
```

C.6 Multi-Panel Overlay Figure

```
% Create a 1x1 figure with measured and theoretical overlaid
figure;
hold on;
plot(time_s, voltage_V, 'b-', 'LineWidth', 1.5, 'DisplayName', 'Measured');
plot(time_s, V_theory, 'r--', 'LineWidth', 2, 'DisplayName', 'Theory');
xlabel('Time (s)', 'FontSize', 12);
ylabel('Voltage (V)', 'FontSize', 12);
legend('FontSize', 11);
title(sprintf('Alice Smith (A123) - RC Step Response'), 'FontSize', 12);
grid on;
hold off;
```

Appendix D: Troubleshooting Guide

Hardware Issues

1. Arduino does not respond to Serial Monitor input.

- Verify USB cable is connected and the Arduino IDE shows the correct COM port.
- Recompile and re-upload the sketch.
- Check that the baud rate in Serial Monitor is set to 115200.

2. Voltage trace is flat at 0 V or does not change.

- Check that A0 is connected to the measurement node (between R and C for RC circuit, or between R_{ext} and L for RL circuit).
- Use a multimeter to verify that the node voltage changes when D7 is toggled manually.
- Verify that the Arduino GND is connected to the breadboard GND rail.
- If the trace is constant but non-zero, the circuit may be stuck in a steady state; check component values.

3. RL circuit: Arduino resets when applying step, or USB disconnects.

- The flyback diode is missing or incorrectly oriented. Verify diode cathode (striped end) points to the measurement node, and anode points to GND.
- Without the diode, the inductor's back-EMF can exceed 5 V and damage the Arduino. Stop immediately and fix the diode.
- Use a multimeter in diode mode to confirm the diode conducts backward (from GND to node).

4. Voltage spikes or ringing on the scope.

- Breadboard layout: keep wires short and avoid loops.
- Verify component tolerances; high capacitive or inductive parasitics can cause ringing.
- This is **non-ideal**, but expected in a breadboard setup. Document it in your discussion.

Arduino Acquisition Issues

1. Serial Monitor shows no data after sending 's'.

- Verify the LED on D13 flashes. If it does not, the Arduino may not be receiving the trigger character. Check Serial Monitor connection and baud rate.
- Check that `INPUT_TYPE` is defined as either `STEP` or `IMPULSE`, not an invalid value.
- Verify `N_SAMPLES` is not too large (max ~30000 for Arduino Uno memory).

2. Data arrives very slowly, or MATLAB hangs waiting for data.

- The sample rate may be too high for the Arduino to keep up. Reduce `SAMPLE_RATE_HZ`.
- Check that serial transmission is at 115200 baud (not a slower rate).
- The Arduino may be overflowing its buffer. Verify `N_SAMPLES` is reasonable (< 10000 for safety).

3. ADC readings jump erratically or show noise spikes.

- This is normal for breadboard circuits. Noise from switching and parasitic capacitance is expected.
- Mitigation: use a smoothing median filter in MATLAB (e.g., `y_smooth = medfilt1(y, 5)`) to remove outliers without introducing lag.
- Verify A0 is not near noisy digital lines (e.g., D7 during switching). Reroute if possible.

MATLAB Analysis Issues

1. MATLAB hangs when calling the serial reader.

- Verify the Arduino is ready and waiting for the 's' trigger.
- Check that MATLAB's serial port name matches the Arduino's COM port (e.g., 'COM3', not 'COM1').
- Ensure Arduino baud rate (115200) matches MATLAB's serial settings.

2. Voltage conversion produces out-of-range values (e.g., $> 5\text{ V}$ or $< 0\text{ V}$).

- The ADC should output 0–1023. If you see values outside this range, there is a data corruption issue.
- Check the serial data in the Serial Monitor: confirm values are in the expected range.
- The Arduino reference voltage may not be 5.0 V. Measure the actual AREF voltage with a multimeter and update the conversion formula.

3. Overlay plot shows measured data but no theoretical curve.

- Verify you computed the theoretical curve using the correct **measured** time constants (τ_{RC} , τ_{RL}), not assumed values.
- For RL circuits, ensure you corrected the A0 voltage to get $V_{R_{\text{ext}}}$ (using $V_{R_{\text{ext}}} = 5.0 - V_{\text{node}}$).

- Check the time vector: if theory uses a different time base than measured data, the curves will not align.

4. **Log-linearization plot is curved, not straight.**

- **Common causes:**

- Baseline offset not removed (especially critical for step responses).
- Impulse pulse is too wide (T_p not $\ll \tau$); the data exhibits more complex dynamics than a pure exponential.
- Strong noise on the measured signal. Median filter before log transform.
- For step responses, you may have selected a fitting window that includes the non-linear transient region.

- **Fix:**

- Carefully remove the baseline: $z = V_{\text{final}} - V_{\text{measured}}$ (for rising steps).
- Select the fitting window from $t \approx 0.2\tau$ to $t \approx 4\tau$ (avoid the very beginning and end).
- Apply a median filter to reduce noise spikes before log transform.

5. **Percent error is $> 30\%$ for both circuits.**

- Verify your component measurements with the multimeter. Resistor and capacitor tolerances can be large ($\pm 10\%$ or more).
- Check that you used the **correct** measured values in your theoretical τ calculation (not nominal values from the component labels).
- For RL circuits, did you include the inductor's winding resistance R_L in R_{eff} ? This is critical.
- Examine the raw Arduino data for signs of noise or ADC quantization. Smooth if needed.
- If RC error is high, check capacitor leakage or parasitic resistance. If RL error is high, re-verify the flyback diode and RL measurement.

6. **RC and RL percent errors are very different (one $< 10\%$, the other $> 30\%$).**

- For RL circuits, the key suspect is the inductor winding resistance R_L . If R_L is measured incorrectly, τ_{RL} will be wrong.
- Re-measure R_L by removing the inductor from the breadboard and measuring it directly with the multimeter.
- For RC circuits, capacitor tolerance can be large. Verify with the multimeter capacitance mode.
- Make sure your curve-fit window is appropriate for both circuits; they may have different optimal windows.

When to Ask for TA Help

- If the Arduino does not respond or Serial Monitor shows no data after 5 minutes of troubleshooting.
- If you suspect hardware damage (Arduino resets, USB disconnects, or components smell hot).
- If measured values are completely outside expected ranges (e.g., $\tau_{RC} > 1$ second when components suggest < 100 ms).
- If you are unsure whether a circuit behavior is normal or an error.

References and Inspiration

This laboratory explores the transient response of first-order analog systems, grounded in classical circuit theory and contemporary embedded signal processing.

University Course Inspirations

MIT 6.007: Applied Electromagnetism and Analog Circuits

Lab 05 draws inspiration from MIT 6.007's treatment of RC and RL circuits as fundamental first-order systems. MIT's approach to analyzing transient response (step and impulse) using classical differential equations and relating them to physical time constants informs the theoretical foundation of this lab.

MIT 6.003: First-Order System Response

MIT's signals and systems course provides the mathematical framework for understanding first-order system response: poles in the s-plane, time constants, settling behavior, and steady-state properties. This lab validates those concepts in hardware.

Georgia Tech ECE 2026: Real-Time System Characterization

Georgia Tech's approach to observing transient response on real hardware—using oscilloscope time measurements and digital recording—informs this lab's emphasis on practical time constant extraction and validation methodology.

Rice University ELEC 241: Embedded Analog Interface Design

The design of ADC-compatible analog circuits and the extraction of system time constants from sampled data reflect Rice's practical approach to interfacing analog systems with digital embedded processors.

Duke University ECE 280: Lab Modernization

This lab is part of Duke's modernized signals and systems sequence emphasizing the bridge between analog circuit theory, continuous-time system response, and discrete-time digital measurement and validation.

References

- A. V. Oppenheim and A. S. Willsky, *Signals and Systems*, 2nd ed. Pearson, 1997. Chapters on differential equations and first-order system response.
- J. H. McClellan, R. W. Schafer, and M. A. Yoder, *DSP First: A Multimedia Approach*, 2nd ed. Pearson, 2015. Chapters on continuous-time systems and step/impulse response.
- A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010. Chapter on discrete-time systems and sample acquisition from continuous-time signals.
- Texas Instruments and ON Semiconductor Op-Amp Application Notes (circuit design and transient characterization).